Experiment 1

System Verilog Modelling and Verification of Combinational and Sequential Logic Blocks

AIM:

To design and implement a Traffic Light Controller with pedestrian request functionality using an Finite State Machine (FSM) in SystemVerilog with enumerated data types and one-hot state encoding, to verify its operation through RTL simulation using Synopsys VCS, and to synthesize the design using Synopsys Design Compiler.

SOFTWARE/TOOLS USED:

Synopsys VCS, Synopsys Design Compiler

THEORY:

An **Finite State Machine (FSM)** is a sequential digital system whose operation is defined by a finite number of states, state transitions, and outputs. FSMs are widely used in control-oriented applications such as traffic light controllers, sequence detectors, vending machines, and communication protocols. An FSM generally consists of a next-state combinational logic, a state-transition logic consisting of flip-flops, and an output combinational logic block.

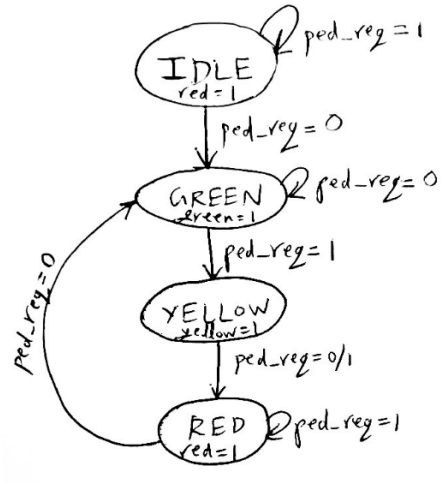
In this experiment, a **traffic light controller with pedestrian request** is designed using **SystemVerilog** and verified through RTL simulation. The controller manages three traffic signals—**Red, Green, and Yellow**—based on a clock input and a pedestrian request signal (ped_req). The design follows a **Moore machine architecture**, where the outputs depend only on the current state of the FSM and not directly on the input signals.

The FSM is implemented using **SystemVerilog enumerated data types** with **one-hot state encoding**. The FSM transitions between states based on the pedestrian request signal. After reset, the system enters the IDLE [0001] state with the red signal asserted. If no pedestrian request is present, the controller moves to the GREEN [0010] state. While in GREEN, the system continues normal traffic flow until a pedestrian request is asserted, upon which it transitions to the YELLOW [0100] warning state. The YELLOW state is maintained for one clock cycle before transitioning to the RED [1000] state. In the RED state, the controller remains stopped as long as the pedestrian request is active and returns to the GREEN state once the request is deasserted.

STATE TRANSITION TABLE

Present state	ped_req	NEXT STATE	OUTPUT
0001(IDLE)	0	0010(GREEN)	RED=1,GREEN=0,YELLOW=0
0001(IDLE)	1	0001(IDLE)	RED=1,GREEN=0,YELLOW=0
0010(GREEN)	0	0010(GREEN)	RED=0,GREEN=1,YELLOW=0
0010(GREEN)	1	0100(YELLOW)	RED=0,GREEN=1,YELLOW=0
0100(YELLOW)	0	1000(RED)	RED=0,GREEN=0,YELLOW=1
0100(YELLOW)	1	1000(RED)	RED=0,GREEN=0,YELLOW=1
1000(RED)	0	0010(GREEN)	RED=1,GREEN=0,YELLOW=0
1000(RED)	1	1000(RED)	RED=1,GREEN=0,YELLOW=0

STATE DIAGRAM



SIMULATION STEPS USING SYNOPSYS VCS

1. The Synopsys environment is initialized by invoking the C shell and sourcing the setup script.
2. The SystemVerilog design file and corresponding testbench file are compiled and simulated using Synopsys VCS with SystemVerilog support and debugging enabled:

3. The simulation executable is executed and waveform data is generated in the form of a .vcd file.
4. The generated waveform file is viewed using the waveform viewer to analyze the state transitions and output signals.

SIMULATION STEPS USING SYNOPSYS DESIGN COMPILER

1. The Synopsys environment is initialized by invoking the C shell and sourcing the setup script.
2. Invoking Design Compiler Shell

The Design Compiler environment is started by invoking the Design Compiler shell from the Linux terminal: dc_shell

3. Launching the Design Compiler GUI

To enable graphical interaction and visualization, the Design Compiler graphical user interface is launched: gui_start

The GUI allows easier viewing of reports, schematics, and design hierarchy.

4. Reading the Design

The SystemVerilog design file describing the FSM is loaded for synthesis.

5. Setting Link, Target, and Symbol Libraries

The required libraries are specified in the Design Compiler environment:

- **Target Library** – Defines the standard cells used for synthesis
- **Link Library** – Used to resolve references during design linking
- **Symbol Library** – Used for schematic and symbol generation

These libraries provide technology-specific information required for logic synthesis.

6. Defining the Design Top Module

The top-level design module is specified to ensure correct synthesis and reporting.

7. Applying Timing Constraints:

Clock definition, Input delays, and Output delays are specified in the dc_shell prompt.These constraints ensure that the synthesized design meets the required timing specifications.

8. Compiling the Design

The design is synthesized by invoking the compile command in Design Compiler. During compilation the RTL description is optimized, logic is mapped to standard cells, timing constraints are considered. This step generates a gate-level representation of the FSM.

9. Viewing the Synthesized Schematic

After successful compilation, the synthesized schematic is generated and viewed using the Design Compiler GUI.

DESIGN FILES:

1. **Source Code (.sv)**

```
//Module Declaration

module traffic_light_encoding(

        input logic clk,

        input logic reset,

        input logic ped_req,

        output logic red,

        output logic green,

        output logic yellow

        );

        //Declaring Enumeration

        typedef enum logic[3:0]{

                IDLE=4'b0001,

                GREEN=4'b0010,

                YELLOW=4'b0100,

                RED=4'b1000

                }state_t;

        state_t ps, ns;

        //Next state logic

        always_comb

        begin

                case(ps)

                        IDLE:

                                ns=GREEN;

                        GREEN:

                        if(ped_req)

                                ns=YELLOW;

                        else

                                ns=GREEN;

                        YELLOW:

                                ns=RED;

                        RED:
```

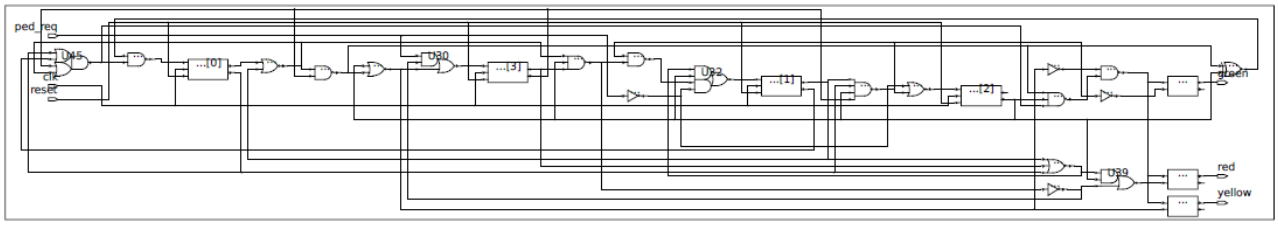
```
                if(ped_req)
                        ns=RED;
                else
                        ns=GREEN;
                default: ns=IDLE;
        endcase
    end
    //State Transition Logic
    always_ff @(posedge clk or negedge reset)
    begin
        if(!reset)
                ps<=IDLE;
        else
                ps<=ns;
    end
    //State machine output logic
    always_comb
    begin
        case(ps)
                IDLE:
                        begin
    red=1'b1;
    green=1'b0;
    yellow=1'b0;
                        end
                GREEN:
                        begin
    red=1'b0;
    green=1'b1;
    yellow=1'b0;
                        end
                YELLOW:
                        begin
    red=1'b0;
    green=1'b0;
    yellow=1'b1;
                        end
                RED:
                        begin
    red=1'b1;
    green=1'b0;
    yellow=1'b0;
```

```
                end
        endcase
    end
endmodule
```

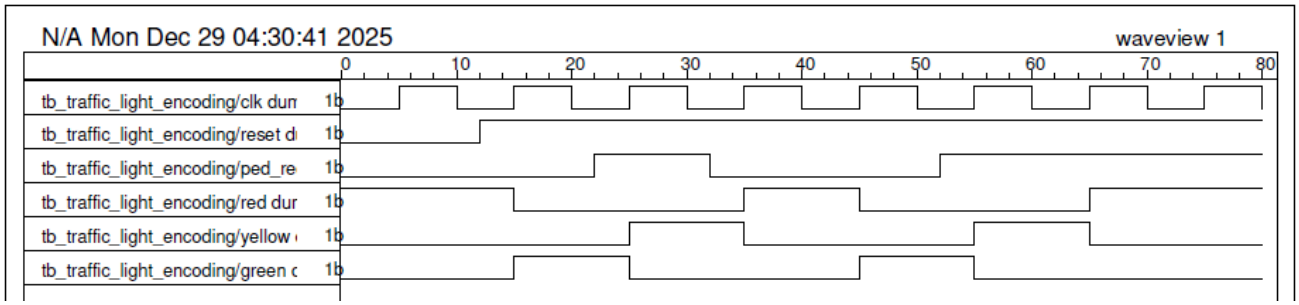
2. **Testbench (.sv)**

```
module tb_traffic_light_encoding;
logic clk;
logic reset;
logic ped_req;
logic red;
logic green;
logic yellow;
//Instantiate device under test
traffic_light_encoding dut(
    .clk(clk),
    .reset(reset),
    .ped_req(ped_req),
    .red(red),
    .green(green),
    .yellow(yellow)
    );
//Clock
always #5 clk=~clk;
initial
begin
    clk=0;
    reset=1'b0;
    ped_req=1'b0;
    $dumpfile("dump6.vcd");
    $dumpvars(0,tb_traffic_light_encoding
);
    #12 reset=1'b1;
    #10 ped_req=1'b1;
    #10 ped_req=1'b0;
    #10 ped_req=1'b0;
    #10 ped_req=1'b1;
    #10 ped_req=1'b1;
    #20;
    $finish;
end
endmodule
```

SCHEMATIC



SIMULATION RESULTS



INFERENCE:

In this experiment, a Traffic Light Controller with pedestrian request was successfully designed using a Finite State Machine (FSM) in SystemVerilog with one-hot state encoding. The design was functionally verified through RTL simulation using Synopsys VCS, and the simulation waveforms confirmed correct state transitions and output behavior in response to the pedestrian request signal. The FSM operated synchronously with the clock and was properly initialized using an asynchronous active-low reset. The design was further synthesized using Synopsys Design Compiler, demonstrating that the FSM is functionally correct, synthesizable, and suitable for implementation in standard-cell–based digital systems.